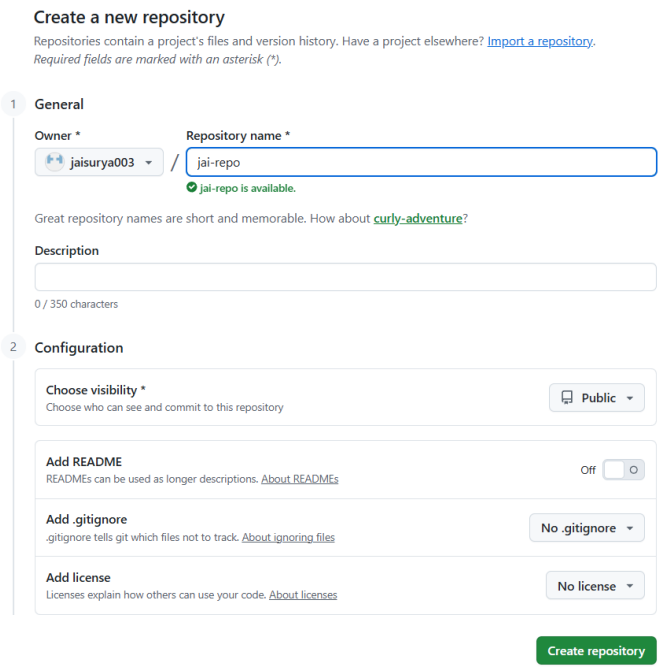
GIT INIT AND REMOTE REPOSITORY SETUP

Git Init - Used to create and initialize a new Git repository in the local system. It creates a hidden .git folder that helps Git track files, commits, and version history in the project directory.

HANDS-ON GIT INIT AND REMOTE REPOSITORY SETUP

Screenshot: Create New Repository

- Open GitHub
- Click New repository
- Enter repository name
- Click Create repository



Screenshot: Initialize Git Repository

- Open Git Bash
- Execute cd Downloads (to move into Downloads folder)
- Execute mkdir repository-name (directory name same as created GitHub repository)
- Execute cd repository-name (to move into project folder)
- Execute git init (to initialize new Git repository)
- Execute ls -la (to view all files and folders)
- .git folder displayed after repository initialization

Screenshot: Add and Commit Files to Repository

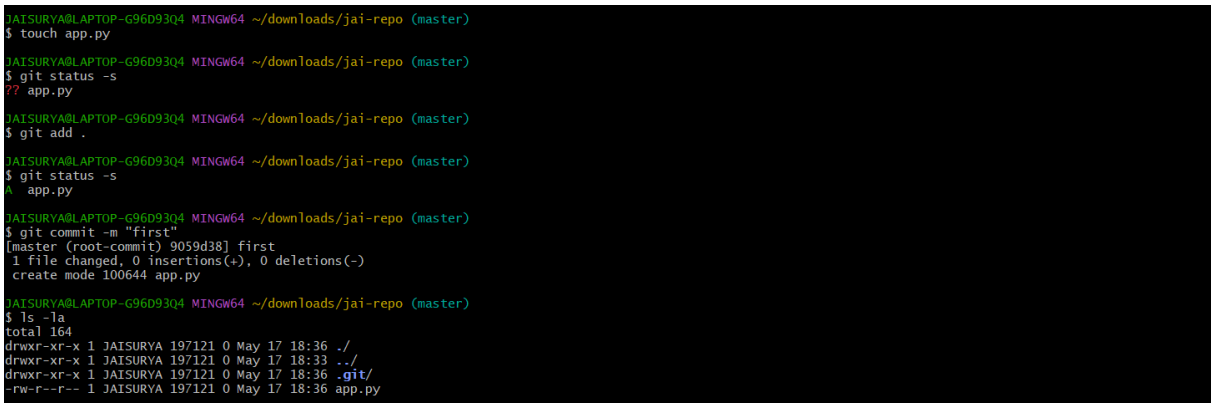
• Execute touch app.py (to create a new Python file)

• Execute git add . (to add files to staging area)

• Execute git status (to check repository status)

• Execute git commit -m "first commit" (to save changes in local repository)

• Execute ls -la (to view all files and folders)



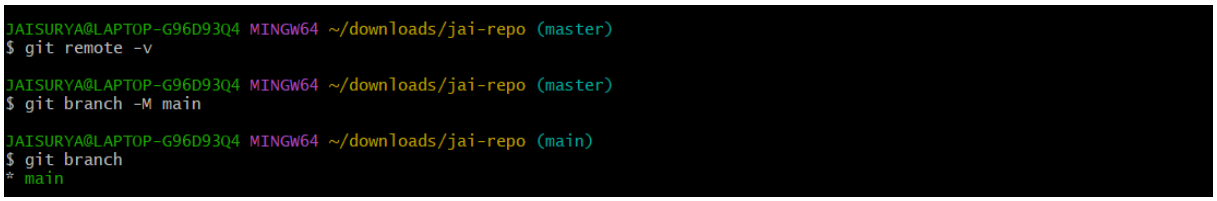
Screenshot: Configure Main Branch

• Execute git remote -v (to check configured remote repositories)

• No current remote repository configured

• Execute git branch -M main (to rename master branch to main branch)

• Execute git branch (to view available branches)



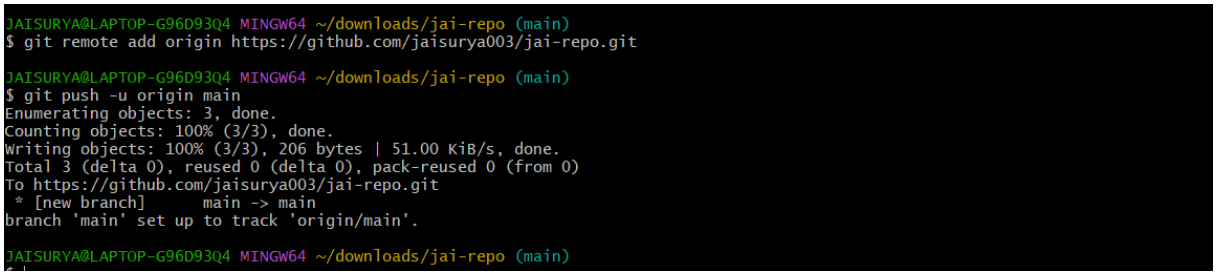
Screenshot: Connect Local Repository to GitHub

• Execute git remote add origin https://github.com/username/repository.git (to connect local repository with GitHub repository)

• Execute git push -u origin main (to push local repository to GitHub main branch)

• Main branch successfully pushed to remote repository

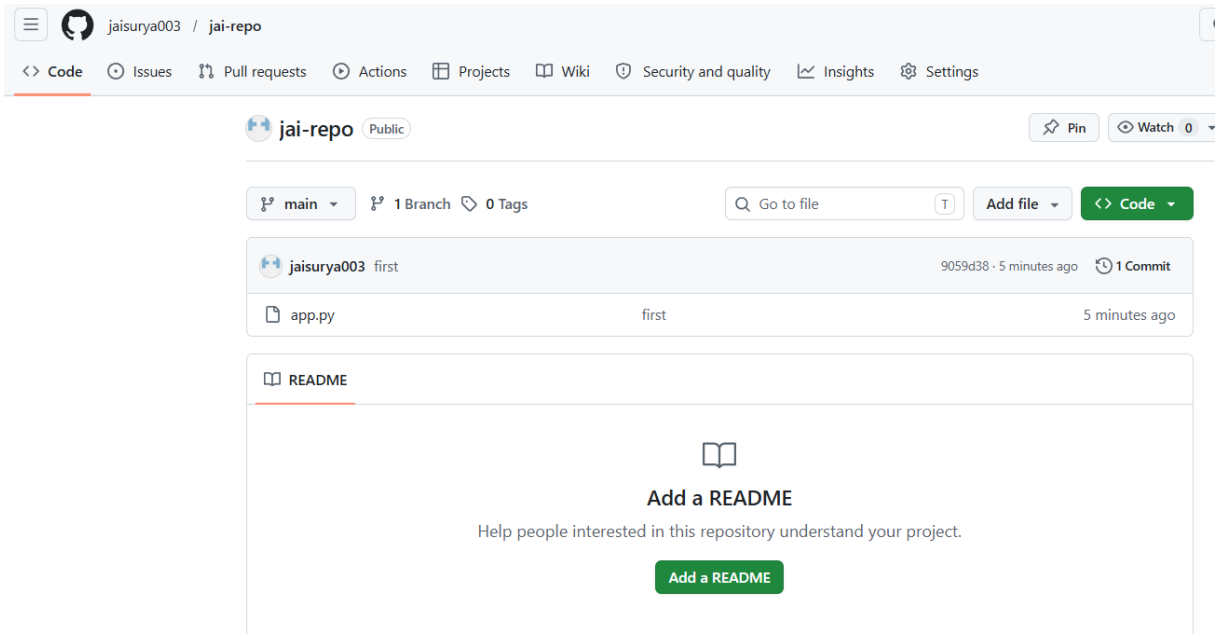
• Local branch connected with remote tracking branch origin/main



Screenshot: Verify Files in GitHub Repository

• Refresh the GitHub repository page

• app.py file displayed successfully in repository



GIT RESET

- Git Reset - Used to undo commits or changes by moving the HEAD pointer to a previous commit in the repository.
- It is mainly used before pushing changes to the remote repository.

Types of Git Reset

1. **git reset --soft <HEAD ID>**: Moves the HEAD to the specified commit, but keeps both the staging area and working directory unchanged.
2. **git reset --mixed <HEAD ID>**: Moves the HEAD to the specified commit and resets the staging area, but keeps the working directory unchanged.
3. **git reset --hard <HEAD ID>**: Moves the HEAD to the specified commit, resets the staging area, and permanently removes changes from the working directory.

HANDS-ON FOR GIT RESET

Screenshot: Create Multiple Commits

• Execute vi app.py (to edit the existing file)

• Execute git status -s (to view short status output)(Red color indicates modified file status)

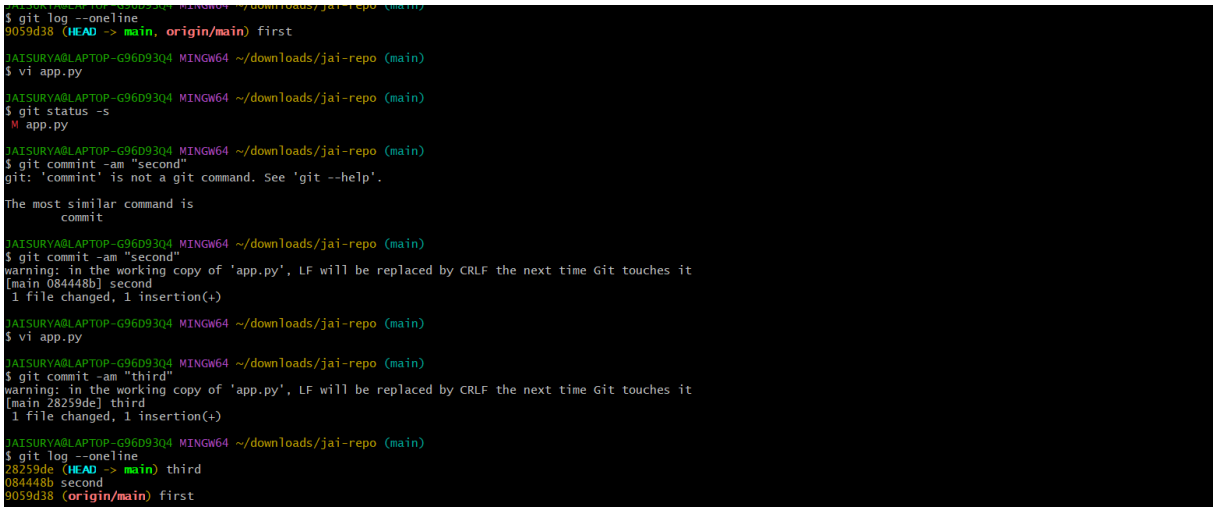
• Execute git add . (to add modified file to staging area)

• Execute git commit -m "second commit" (to save second change in repository)

• Execute vi app.py (to modify the file again)

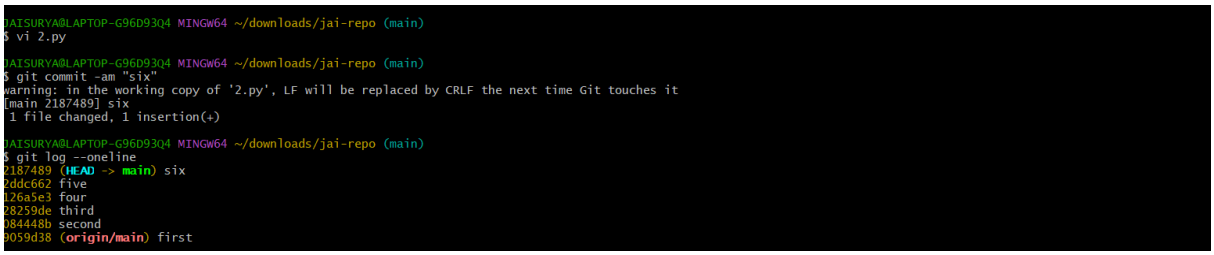
• Execute git add . (to add modified changes to staging area)

• Execute git commit -m "third commit" (to save third change in repository)



Screenshot: Create Multiple Commits

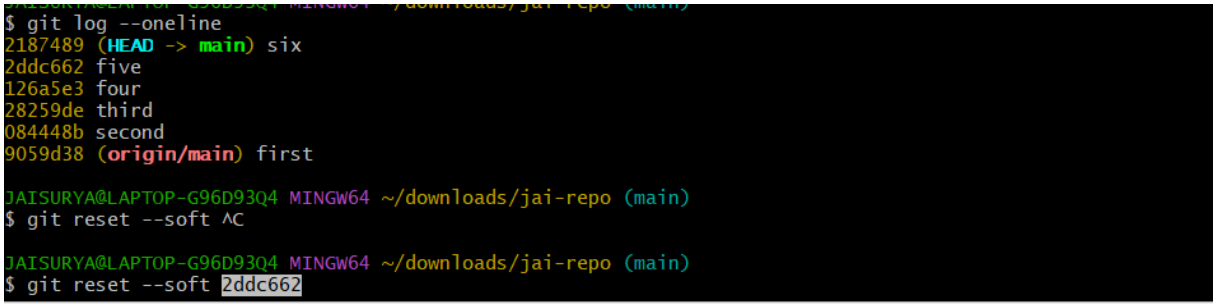
Repeat the same process to create 6 commits in the repository



Screenshot: Perform Soft Reset

• Execute git log --oneline (to view commit history in one line format)

• Execute git reset --soft <fifth-commit-id> (to move HEAD to fifth commit and keep changes in staging area)

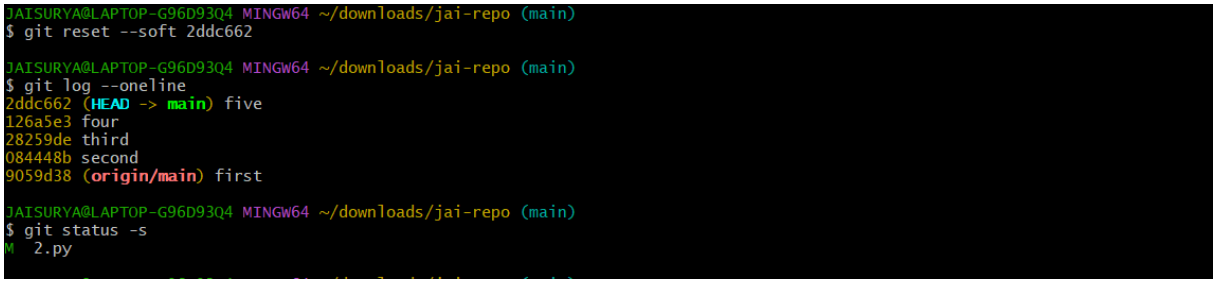


Screenshot: Verify Soft Reset Changes

• HEAD moved from sixth commit to fifth commit

• Execute git status (to check repository status)

• Changes remain unchanged in staging area after soft reset



Screenshot: Perform Mixed Reset

• Execute git log --oneline (to view commit history in one line format)

• Execute git reset --mixed <fourth-commit-id> (to move HEAD to fourth commit and remove changes from staging area)

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
2187489 (HEAD -> main) six
2ddc662 five
126a5e3 four
28259de third
084448b second
9059d38 (origin/main) first

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git reset --mixed 126a5e3
```

Screenshot: Verify Mixed Reset Changes

• HEAD moved from sixth commit to fourth commit

• Execute git status -s (to view short status output)

• Red color indicates modified unstaged changes after mixed reset

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
126a5e3 (HEAD -> main) four
28259de third
084448b second
9059d38 (origin/main) first

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git status -s
?? 2.py
```

Screenshot: Perform Hard Reset

• Execute git log --oneline (to view commit history in one line format)

• Execute git reset --hard <third-commit-id> (to move HEAD to third commit and remove all changes permanently)

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
126a5e3 (HEAD -> main) four
28259de third
084448b second
9059d38 (origin/main) first

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git reset --hard 28259de
```

Screenshot: Verify Hard Reset Changes

• HEAD moved to third commit

• Execute git status (to check repository status)

• No red color displayed after hard reset because changes were permanently removed

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git reset --hard 28259de
HEAD is now at 28259de third

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
28259de (HEAD -> main) third
084448b second
9059d38 (origin/main) first

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git status -s
?? 2.py
```

GIT REFLOG

Git Reflog - Used to view the history of HEAD movements and recent Git actions such as commits, resets, checkouts, and merges in the repository.

```
JAISURYA@LAPTOP-G96DJ3O4 MINGW64 ~/Downloads/jai-repo (main)
$ git reflog --oneline
084448b (HEAD -> main) HEAD@{0}: reset: moving to 084448b
28259de HEAD@{1}: reset: moving to 28259de
126a5e3 HEAD@{2}: reset: moving to 126a5e3
2187489 HEAD@{3}: reset: moving to 2187489
2ddc662 HEAD@{4}: reset: moving to 2ddc662
2187489 HEAD@{5}: commit: six
2ddc662 HEAD@{6}: commit: five
126a5e3 HEAD@{7}: commit: four
28259de HEAD@{8}: commit: third
084448b (HEAD -> main) HEAD@{9}: commit: second
9059d38 (origin/main) HEAD@{10}: Branch: renamed refs/heads/master to refs/heads/main
9059d38 (origin/main) HEAD@{12}: commit (initial): first
```

GIT FORK

- Git Fork - Used to create a personal copy of another user's GitHub repository into your own GitHub account for independent development and collaboration.
- It allows users to modify, experiment, and contribute to projects independently without affecting the original repository.
- Forking is commonly used in open-source collaboration and pull request workflows.

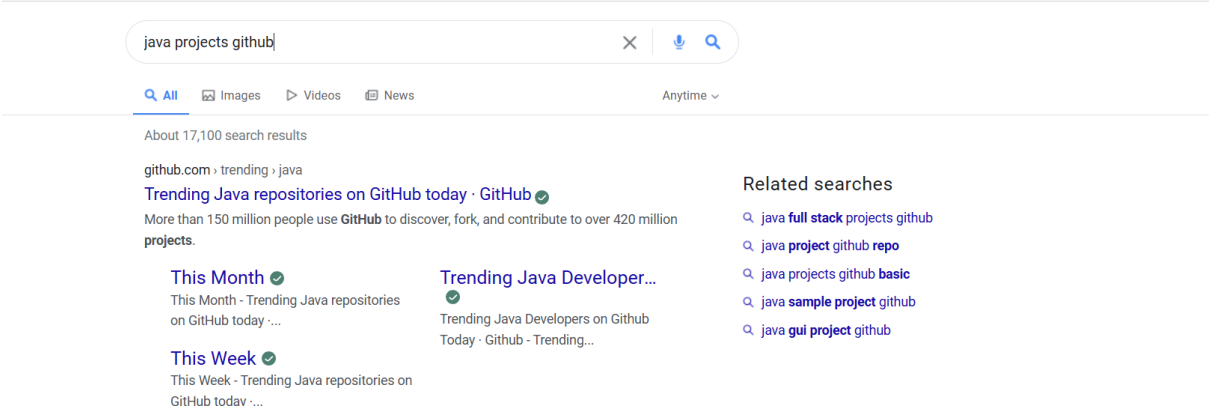
HANDS-ON FOT FORK

Screenshot: Search GitHub Java Project

• Open Google

• Search for Java project in GitHub

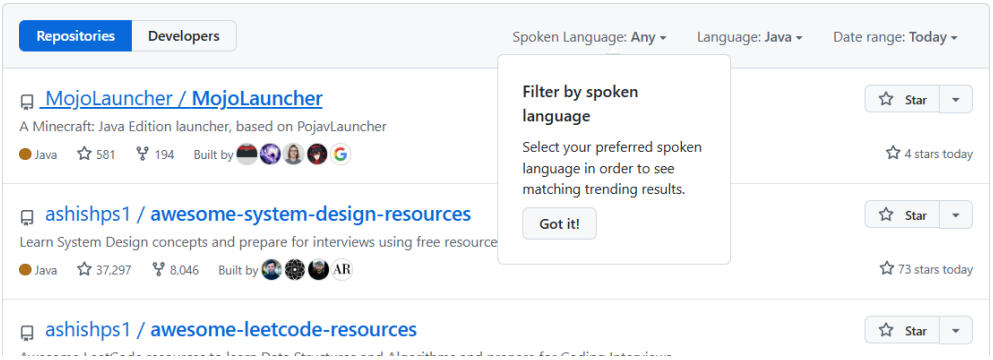
• Open a public GitHub Java project repository



Screenshot: Open Public GitHub Repository

• Open any public GitHub account

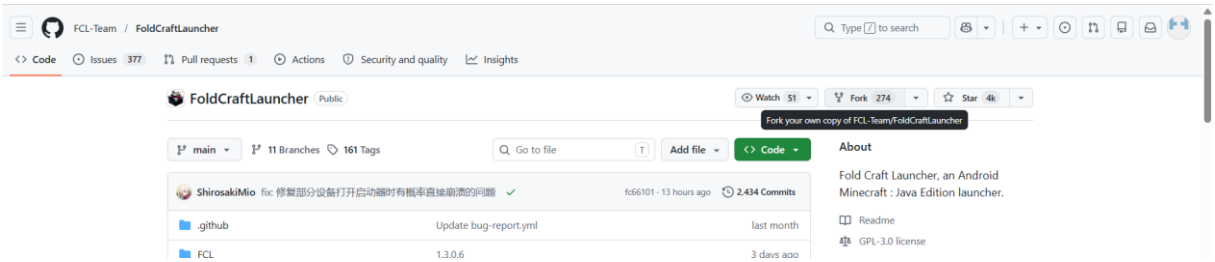
• View available repositories in the account



Screenshot: Fork Repository

• Click Fork

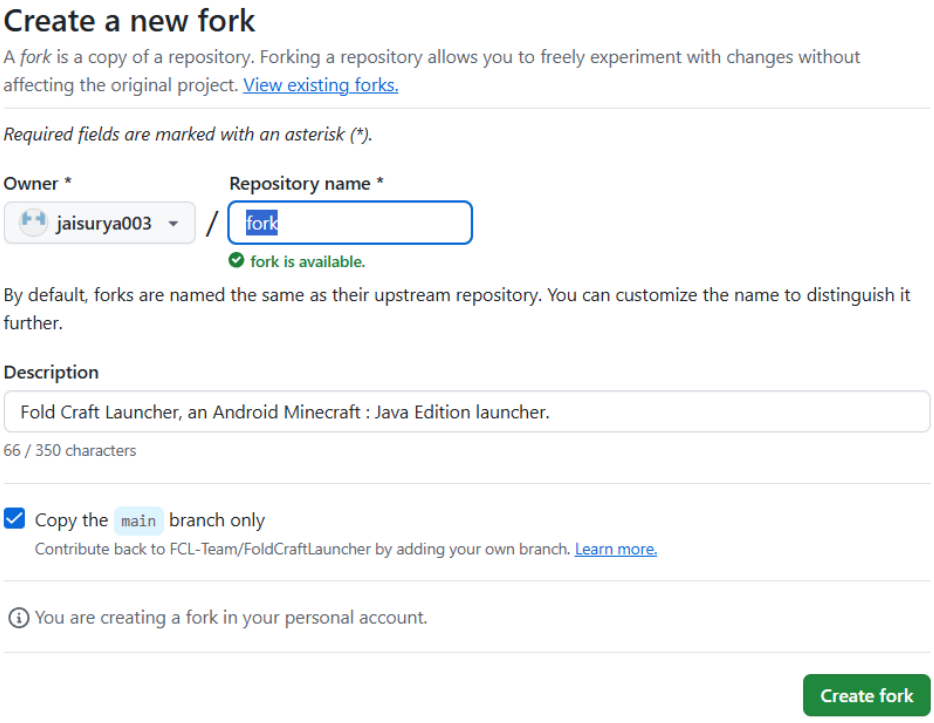
• Start creating a copy of the repository into personal GitHub account



Screenshot: Create Forked Repository

• Enter repository name

• Click Fork



Screenshot: Forked Repository Created Successfully

• Repository copied successfully into personal GitHub account

• Forked repository available for independent development and changes